🗺️ Entity: Mobile

Attributes:

- id
- brandName
- modelName
- price
- ram
- storage
- color

🎯 Objective

Design and implement a **Spring Boot REST API** for managing Mobile information.
The application should demonstrate usage of:

- **DTO (Data Transfer Object)**
- **ResponseEntity** for standardized responses
- **Validation** using Bean Validation annotations
- **Cache** for optimizing read operations
- **Swagger** for API documentation
- **Actuator** for monitoring
- **Global Exception Handling**

🧱 Project Structure

```
com.example.mobileapp
 ├ 📁 entity
 │  └ Mobile.java
 ├ 📁 dto
 │  └ MobileDTO.java
 ├ 📁 repository
 │  └ MobileRepository.java
 ├ 📁 service
 │  ├ MobileService.java
 │  └ MobileServiceImpl.java
 ├ 📁 controller
 │  └ MobileController.java
 ├ 📁 exception
```

```
│  ├ GlobalExceptionHandler.java
│  └ ResourceNotFoundException.java
├ 📄 MobileApplication.java
```

🌐 Endpoints and Method Details

# Endpoint	HTTP Method	Description	Example Usage
1 /api/mobiles	**POST**	Add a new mobile using ResponseEntity and @Valid DTO	Send MobileDTO JSON in request body
2 /api/mobiles	**GET**	Fetch all mobiles (with caching enabled using @Cacheable)	/api/mobiles
3 /api/mobiles/{id}	**GET**	Fetch mobile by ID (return using ResponseEntity)	/api/mobiles/1
4 /api/mobiles/{id}	**PUT**	Update mobile details using DTO	/api/mobiles/1 + MobileDTO body
5 /api/mobiles/{id}	**DELETE**	Delete mobile by ID and clear cache using @CacheEvict	/api/mobiles/1
6 /api/mobiles/search	**GET**	Search mobiles by brand name	/api/mobiles/search?brandName=Samsung

---

⚙️ Implementation Requirements

1️⃣ DTO and Validation

- Create a MobileDTO class with appropriate validation annotations such as @NotBlank, @Min, etc.
- Use @Valid in controller methods to validate incoming data.

2️⃣ ResponseEntity

- Use ResponseEntity to return responses with proper HTTP status codes (200 OK, 201 CREATED, 404 NOT FOUND, etc.).

3️⃣ Caching

- Enable caching with @EnableCaching in the main class.
- Use @Cacheable on the getAllMobiles() method and @CacheEvict on delete operations.

4️⃣ Swagger Integration

- Integrate **Swagger (OpenAPI)** using the springdoc-openapi dependency.
- Verify the Swagger UI at:
  👉 http://localhost:8080/swagger-ui/index.html

5️⃣ Actuator Integration

- Add the Actuator dependency for health and metrics monitoring.
- Enable endpoints by adding:

management.endpoints.web.exposure.include=*

- Verify endpoints such as:
  - /actuator/health
  - /actuator/info
  - /actuator/metrics

6️⃣ Global Exception Handling

- Create a custom exception ResourceNotFoundException.
- Implement GlobalExceptionHandler using @RestControllerAdvice to handle exceptions like:
  - Resource not found
  - Validation errors
  - Generic exceptions